

WebRTC 이론 공부 자료

이 문서는 WebRTC를 처음 공부하는 팀원이 핵심 개념을 이해할 수 있도록 정리한 자료입니다.

반드시 이해해야 하는 핵심 주제:

- P2P 통신
- Signaling 서버
- RTCPeerConnection
- Offer / Answer
- ICE Candidate
- STUN / TURN

1. WebRTC란?

WebRTC는 Web Real-Time Communication의 줄임말입니다.

브라우저에서 별도 프로그램 설치 없이 실시간으로 다음 데이터를 주고받을 수 있게 해주는 기술입니다.

- 화상
- 음성
- 화면 공유
- 파일
- 채팅 데이터

대표적인 사용 예:

- 화상 회의
- 1:1 영상 통화
- 화면 공유 발표
- 실시간 음성 채팅
- 파일 전송

핵심 문장:

WebRTC는 브라우저끼리 실시간으로 영상, 음성, 데이터를 주고받게 해주는 웹 표준 기술입니다.

2. 일반 웹 통신과 WebRTC 통신 차이

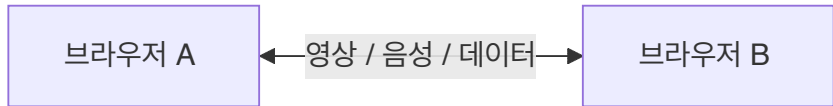
일반적인 웹 서비스는 대부분 브라우저와 서버가 통신합니다.



예를 들어 게시물 목록을 볼 때:

브라우저 → 서버에 게시물 요청
서버 → 게시물 데이터 응답

하지만 WebRTC는 연결이 완료되면 브라우저끼리 직접 데이터를 주고받을 수 있습니다.



정리:

일반 웹 통신

→ 브라우저와 서버가 통신

WebRTC 통신

→ 연결 후 브라우저와 브라우저가 직접 통신

3. P2P란?

P2P는 Peer-to-Peer의 줄임말입니다.

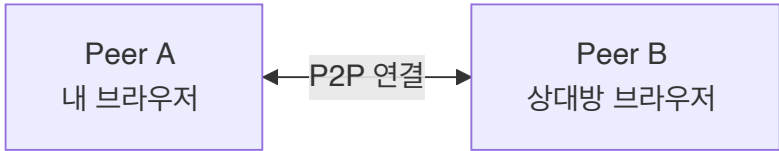
여기서 Peer는 통신에 참여하는 각각의 사용자를 의미합니다.

WebRTC에서 Peer는 보통 브라우저입니다.

Peer A = 내 브라우저

Peer B = 상대방 브라우저

P2P 통신은 중앙 서버가 모든 데이터를 대신 전달하는 방식이 아니라, 참여자끼리 직접 데이터를 주고받는 방식입니다.



P2P의 장점:

- 서버가 영상/음성 데이터를 직접 처리하지 않아도 된다.
- 1:1 통화에서는 구조가 단순하다.
- 지연 시간이 줄어 들 수 있다.

P2P의 단점:

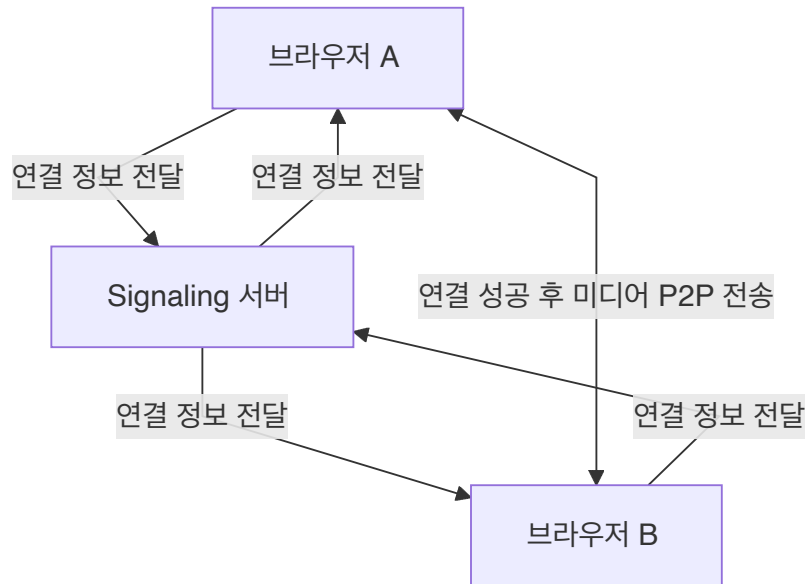
- NAT, 공유기, 방화벽 환경에 따라 연결이 어려울 수 있다.
- 다자간 통화에서는 연결 수가 많아져 부담이 커진다.
- 연결 실패 시 TURN 같은 중계 서버가 필요할 수 있다.

4. WebRTC에서 서버가 아예 필요 없는가?

아닙니다.

WebRTC는 연결 후에는 P2P로 데이터를 주고받을 수 있지만, 연결을 시작하기 전에는 서로의 정보를 교환할 방법이 필요합니다.

이때 사용하는 서버가 Signaling 서버입니다.



중요한 점:

Signaling 서버는 영상과 음성을 직접 중계하는 서버가 아닙니다. WebRTC 연결을 만들기 위한 정보를 전달하는 서버입니다.

5. Signaling 서버란?

Signaling 서버는 WebRTC 연결을 만들기 위해 브라우저 사이에서 연결 정보를 전달하는 서버입니다.

Signaling 서버가 전달하는 대표적인 정보:

- Offer
- Answer
- ICE Candidate
- 방 입장 정보
- 사용자 나가기 정보

우리 프로젝트에서는 `server.js` 가 Signaling 서버 역할을 합니다.



Signaling 서버는 WebRTC 표준에 포함되어 있지 않습니다.

즉, WebRTC는 "연결 정보가 필요하다"는 구조만 정해두고, 그 정보를 어떻게 주고받을지는 개발자가 직접 구현해야 합니다.

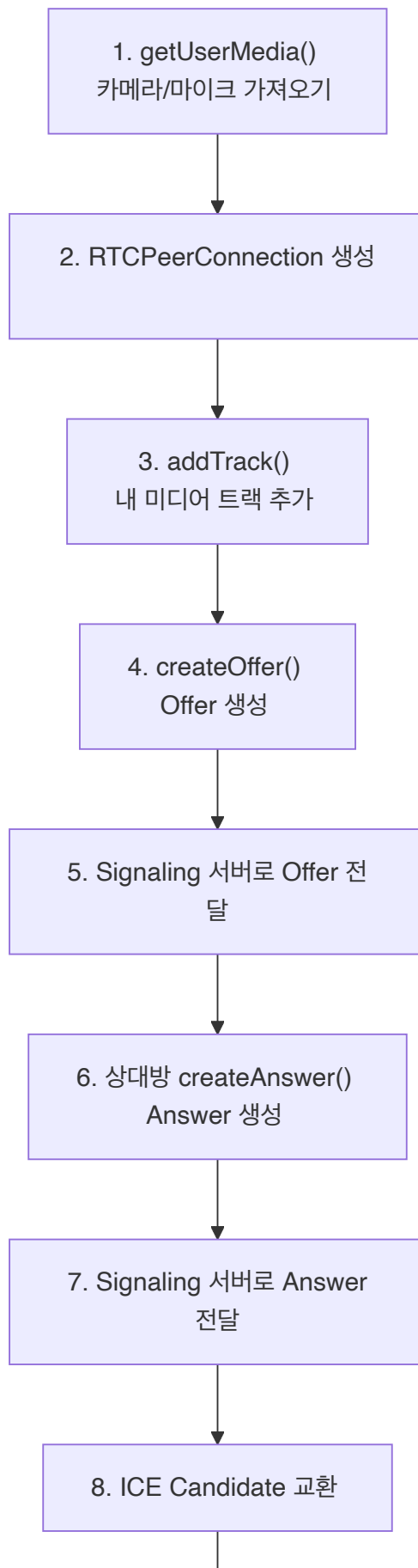
보통 사용하는 방식:

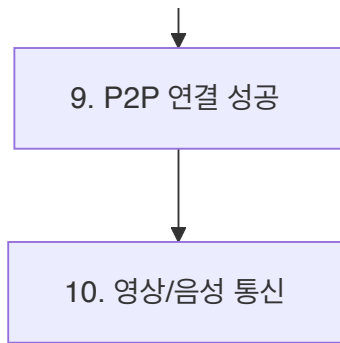
- WebSocket
- Socket.IO
- HTTP polling
- Firebase 같은 실시간 DB

우리 프로젝트에서는 Socket.IO를 사용했습니다.

6. WebRTC 전체 연결 흐름

WebRTC 연결은 크게 아래 순서로 진행됩니다.





짧게 정리하면:

미디어 준비
→ 연결 객체 생성
→ offer / answer 교환
→ ICE candidate 교환
→ P2P 연결

7. RTCPeerConnection이란?

`RTCPeerConnection`은 WebRTC의 가장 핵심적인 객체입니다.

브라우저와 브라우저 사이의 연결을 만들고 관리합니다.

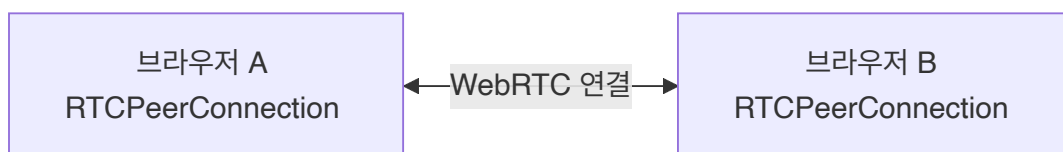
역할:

- 내 영상/음성 트랙을 연결에 추가
- 상대방 영상/음성 트랙 수신
- Offer 생성
- Answer 생성
- ICE Candidate 수집
- 연결 상태 관리

기본 코드:

```
const peerConnection = new RTCPeerConnection({
  iceServers: [
    { urls: "stun:stun.l.google.com:19302" }
  ]
});
```

그림으로 보면:



중요한 점:

WebRTC에서 실제 연결을 담당하는 객체는 `RTCPeerConnection`입니다. Signaling 서버는 연결 정보를 전달할 뿐이고, 실제 미디어 연결은 `RTCPeerConnection`이 관리합니다.

8. MediaStream과 Track

WebRTC에서 카메라와 마이크는 `MediaStream`으로 가져옵니다.

```
const localStream = await navigator.mediaDevices.getUserMedia({
  video: true,
  audio: true
});
```

`MediaStream` 안에는 여러 `Track`이 들어 있습니다.

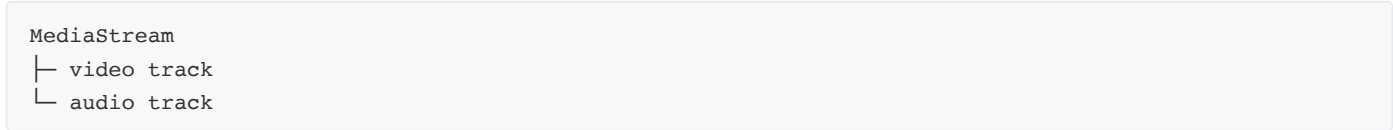
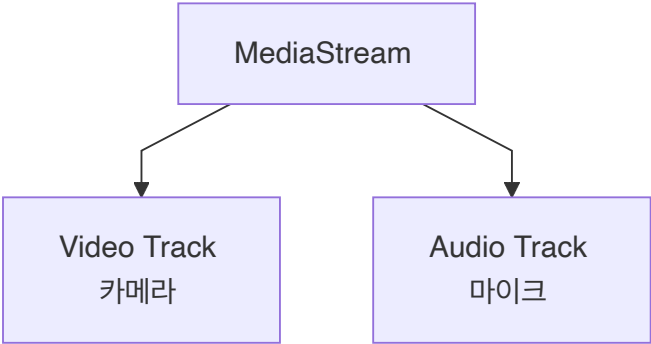


그림:



이 트랙을 `RTCPeerConnection`에 추가해야 상대방에게 보낼 수 있습니다.

```
localStream.getTracks().forEach((track) => {
  peerConnection.addTrack(track, localStream);
});
```

설명:

`getUserMedia()`로 가져온 카메라와 마이크는 자동으로 상대방에게 전달되지 않습니다. `addTrack()`을 통해 `RTCPeerConnection`에 추가해야 WebRTC 연결에 포함됩니다.

9. Offer란?

Offer는 연결을 시작하는 쪽이 만드는 연결 제안서입니다.

Offer 안에는 SDP라는 정보가 들어 있습니다.

Offer에 포함되는 정보:

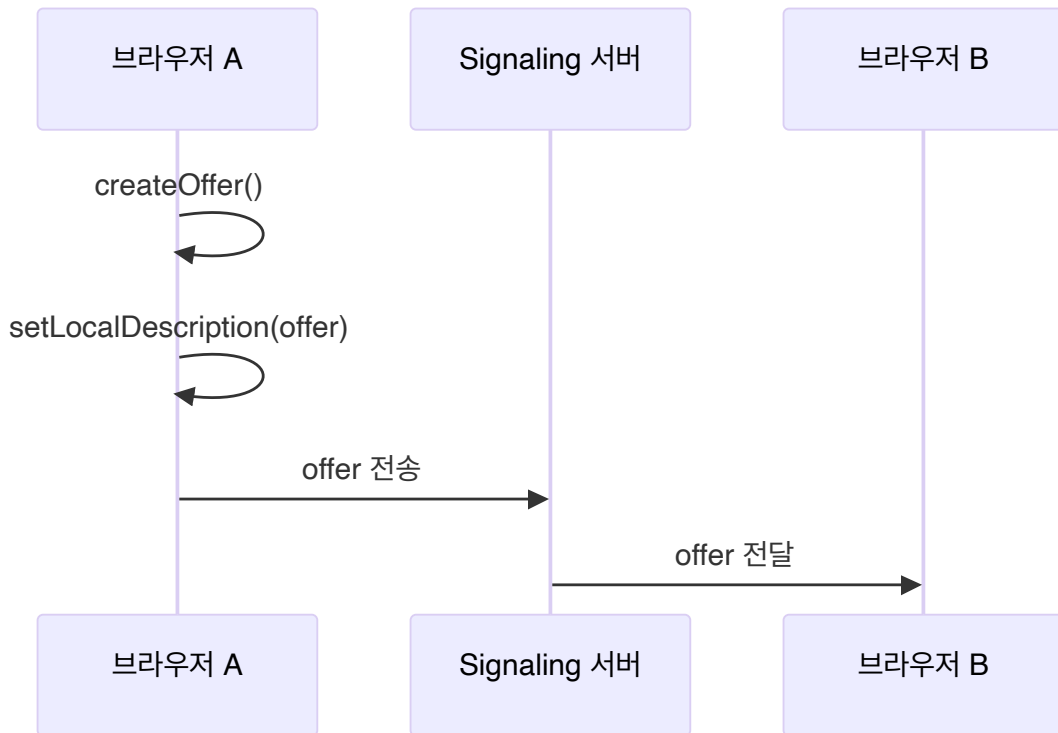
- 내가 보낼 수 있는 미디어 종류
- 오디오/비디오 코덱 정보
- 네트워크 연결 정보
- 암호화 관련 정보

기본 코드:

```
const offer = await peerConnection.createOffer();
await peerConnection.setLocalDescription(offer);

socket.emit("offer", {
  roomId,
  offer
});
```

흐름:



중요한 순서:

```
createOffer()
→ setLocalDescription(offer)
→ signaling 서버로 offer 전송
```

스터디 설명:

Offer는 "나는 이런 방식으로 영상과 음성을 보낼 준비가 되어 있어"라는 연결 제안입니다.

10. Answer란?

Answer는 Offer를 받은 쪽이 만드는 응답입니다.

Offer를 받은 브라우저는 먼저 상대방 Offer를 적용합니다.

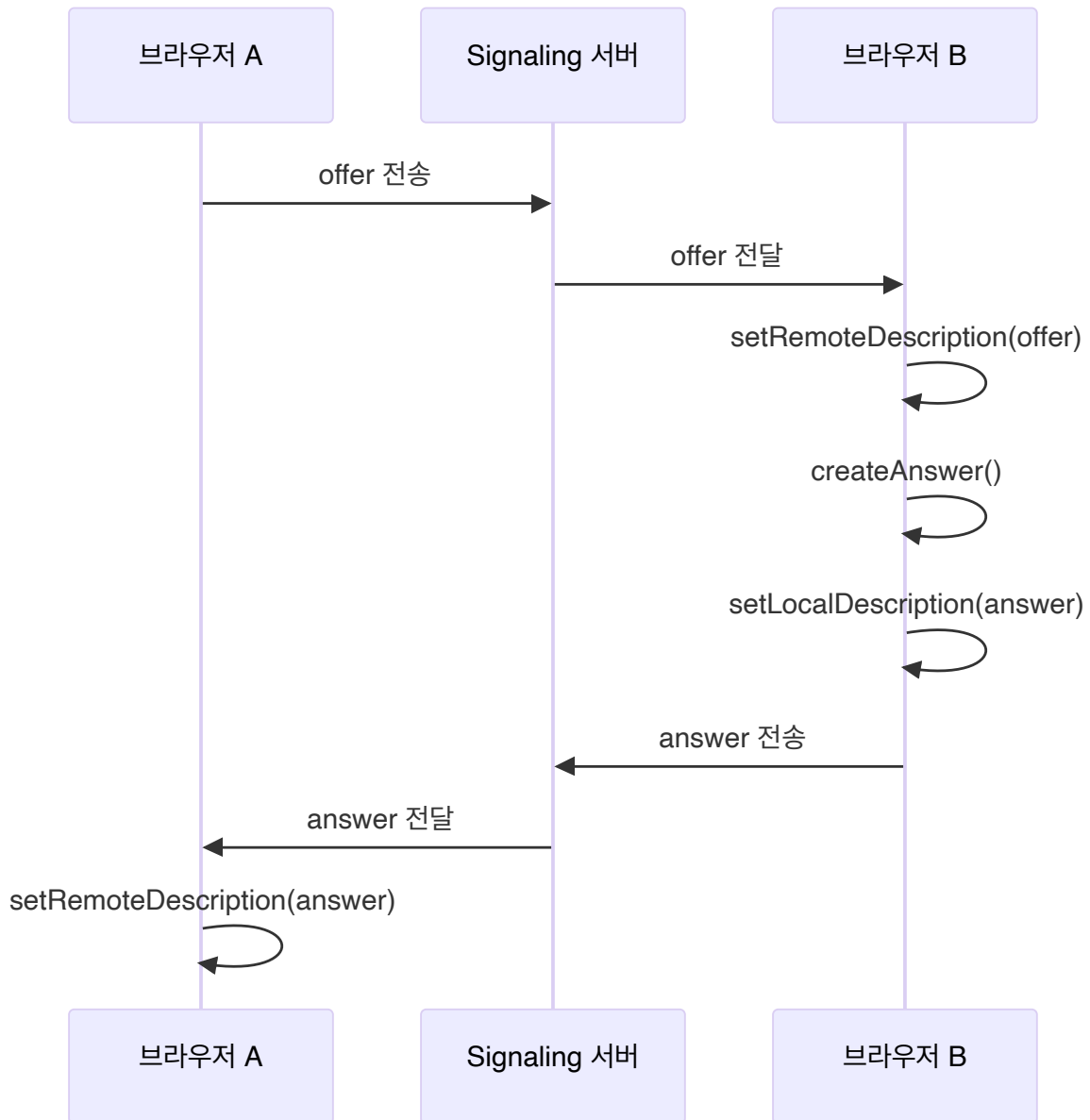
```
await peerConnection.setRemoteDescription(offer);
```

그 다음 Answer를 만듭니다.

```
const answer = await peerConnection.createAnswer();
await peerConnection.setLocalDescription(answer);

socket.emit("answer", {
  roomId,
  answer
});
```

흐름:



중요한 순서:

```
setRemoteDescription(offer)
→ createAnswer()
→ setLocalDescription(answer)
→ signaling 서버로 answer 전송
```

스터디 설명:

Answer는 Offer에 대한 응답입니다. 두 브라우저는 Offer와 Answer를 통해 어떤 방식으로 통신할지 합의합니다.

11. SDP란?

SDP는 Session Description Protocol의 줄임말입니다.

Offer와 Answer 안에 들어 있는 실제 연결 설명 정보입니다.

SDP에는 이런 내용이 들어 있습니다.

- 오디오를 쓸지
- 비디오를 쓸지
- 어떤 코덱을 쓸지
- 어떤 암호화 정보를 쓸지
- 어떤 미디어 라인을 사용할지

개발자가 SDP 문자열을 직접 작성하는 경우는 많지 않습니다.

보통은 브라우저가 만들어줍니다.

```
const offer = await peerConnection.createOffer();
```

그리고 개발자는 이 값을 상대방에게 전달합니다.

정리:

```
Offer / Answer = 연결 제안과 응답 객체  
SDP = Offer / Answer 안에 들어 있는 세부 연결 설명
```

12. ICE Candidate란?

Offer와 Answer만 교환한다고 연결이 끝나는 것은 아닙니다.

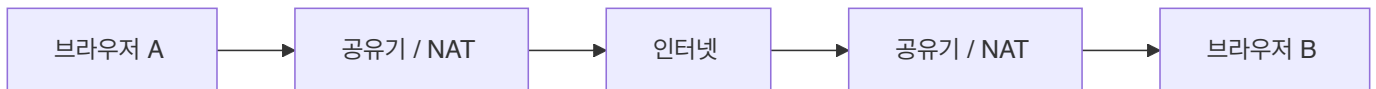
브라우저끼리 실제로 통신할 수 있는 네트워크 경로를 찾아야 합니다.

이때 사용하는 것이 ICE Candidate입니다.

ICE Candidate는 쉽게 말하면:

"나에게 연결하려면 이 주소와 경로로 시도해볼 수 있어"라는 네트워크 후보입니다.

그림:

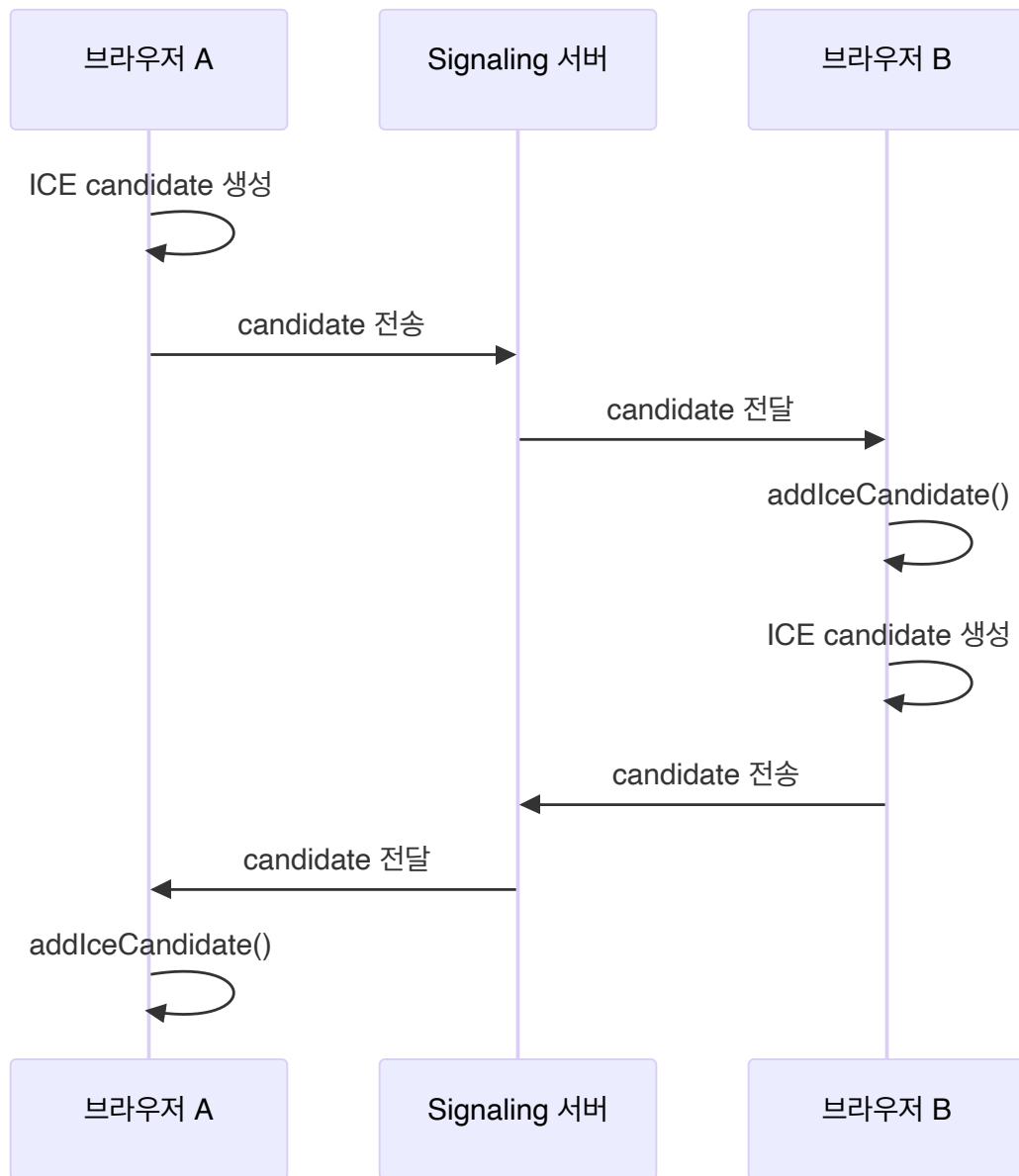


브라우저는 가능한 연결 후보를 여러 개 찾습니다.

후보 예:

```
host candidate  
srflx candidate  
relay candidate
```

ICE Candidate 교환 흐름:



보내는 코드:

```
peerConnection.onicecandidate = (event) => {
  if (event.candidate) {
    socket.emit("ice-candidate", {
      roomId,
      candidate: event.candidate
    });
  }
};
```

받는 코드:

```
await peerConnection.addIceCandidate(candidate);
```

스터디 설명:

ICE Candidate는 두 브라우저가 실제로 연결 가능한 네트워크 경로 후보입니다. 후보는 하나가 아니라 여러 개 생길 수 있고, 양쪽 브라우저가 이 후보들을 서로 교환하면서 가장 적절한 경로를 찾습니다.

13. ICE Candidate 종류

Host Candidate

내 기기의 로컬 네트워크 주소입니다.

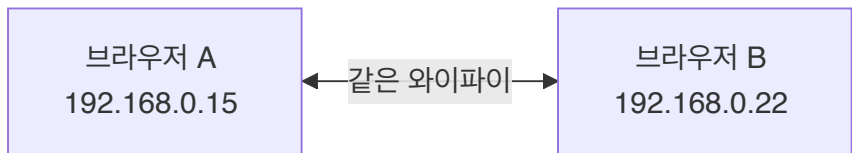
예:

```
192.168.0.15
10.0.0.5
```

특징:

- 별도 서버 없이 브라우저가 자동 생성
- 같은 네트워크 안에서는 연결될 수 있음
- 외부 인터넷에서는 보통 직접 접근 불가

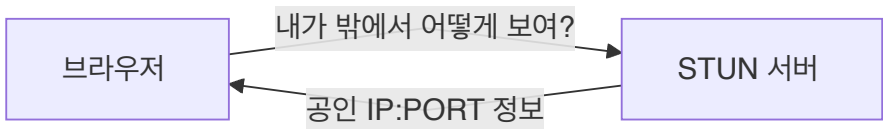
그림:



Server Reflexive Candidate

STUN 서버를 통해 알게 된 외부 주소 후보입니다.

보통 `srflix` 라고 표시됩니다.



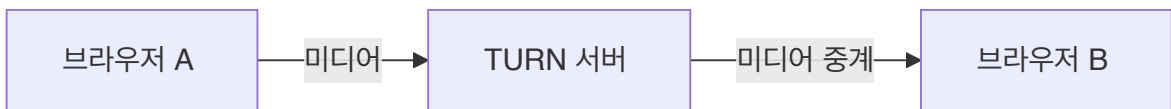
특징:

- STUN 서버가 필요
- 외부 네트워크 간 직접 연결에 사용될 수 있음
- NAT 환경에 따라 실패할 수 있음

Relay Candidate

TURN 서버를 통해 중계하는 후보입니다.

보통 `relay` 라고 표시됩니다.



특징:

- TURN 서버가 필요

- 직접 P2P 연결이 안 될 때 사용
- 가장 안정적이지만 서버 트래픽 비용이 높음

정리:

```
host = 내 로컬 주소 후보
srflx = STUN으로 얻은 외부 주소 후보
relay = TURN 서버 중계 후보
```

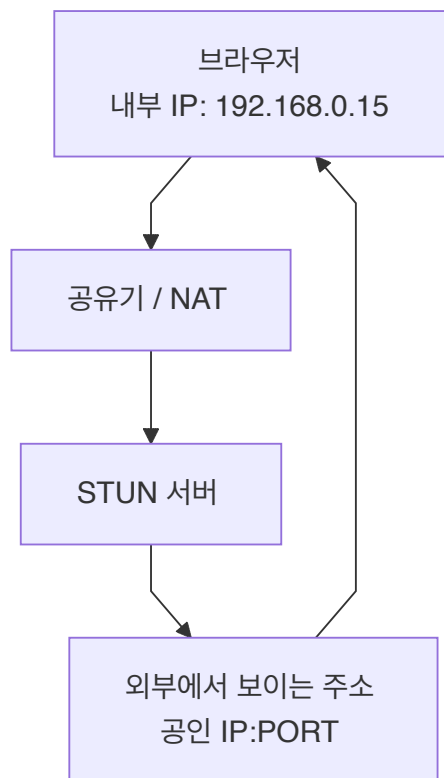
14. STUN 서버란?

STUN은 Session Traversal Utilities for NAT의 줄임말입니다.

브라우저가 NAT나 공유기 뒤에 있을 때, 외부에서 보이는 자신의 IP와 포트를 알아내는 데 사용합니다.

```
const peerConnection = new RTCPeerConnection({
  iceServers: [
    { urls: "stun:stun.l.google.com:19302" }
  ]
});
```

그림:



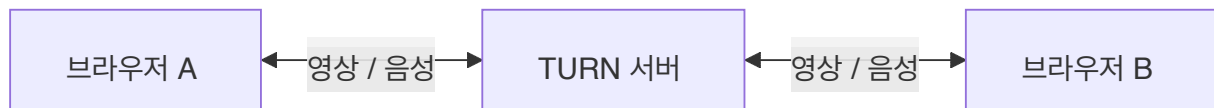
설명:

STUN 서버는 내 브라우저가 외부에서 어떤 주소로 보이는지 알려줍니다. 이 정보는 srflx candidate로 사용됩니다.

15. TURN 서버란?

TURN은 Traversal Using Relays around NAT의 줄임말입니다.

브라우저끼리 직접 연결할 수 없을 때, TURN 서버가 중간에서 데이터를 대신 전달합니다.



TURN이 필요한 경우:

- 회사 네트워크
- 학교 네트워크
- 공공 와이파이
- 방화벽이 강한 환경
- Symmetric NAT
- STUN만으로 연결 실패하는 환경

TURN 설정 예:

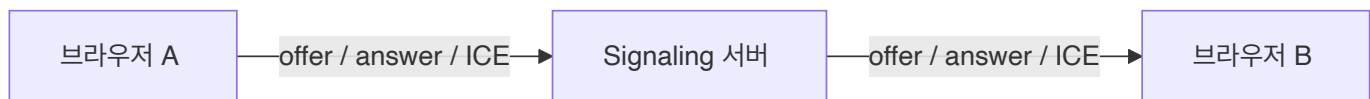
```
const peerConnection = new RTCPeerConnection({
  iceServers: [
    { urls: "stun:stun.l.google.com:19302" },
    {
      urls: "turn:your-turn-server.com:3478",
      username: "turn-user",
      credential: "turn-password"
    }
  ]
});
```

설명:

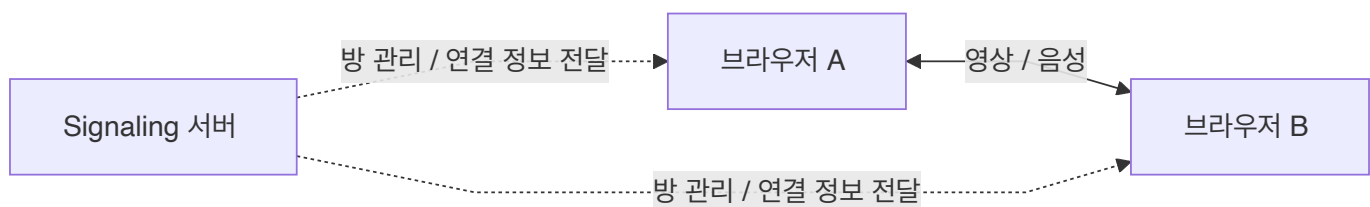
TURN 서버는 직접 연결이 안 될 때 사용하는 중계 서버입니다. 안정적이지만 실제 영상과 음성이 서버를 지나가기 때문에 트래픽 비용이 발생합니다.

16. 연결 전과 연결 후 데이터 흐름

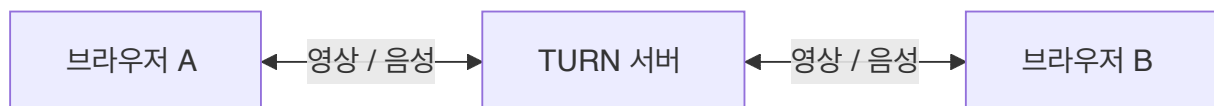
연결 전에는 Signaling 서버를 통해 연결 정보가 이동합니다.



연결 후에는 영상과 음성이 P2P로 이동합니다.



직접 연결이 실패하면 TURN 서버를 통해 이동합니다.



17. 우리 프로젝트와 연결해서 보기

우리 프로젝트 파일 역할:

```
server.js
→ Signaling 서버
→ offer / answer / ice-candidate 전달

public/client.js
→ WebRTC 실제 구현
→ RTCPeerConnection 생성
→ getUserMedia 처리
→ offer / answer 처리
→ ICE Candidate 처리

public/index.html
→ 화면 구조

public/style.css
→ 화면 스타일
```

우리 프로젝트에서 WebRTC 핵심 코드는 `public/client.js`에 있습니다.

핵심 코드 위치:

```
startLocalMedia()
→ getUserMedia()로 카메라/마이크 가져오기

createPeerConnection()
→ RTCPeerConnection 생성
→ addTrack()
→ onicecandidate
→ ontrack

createAndSendOffer()
→ createOffer()
→ setLocalDescription()
→ offer 전송

handleOffer()
→ setRemoteDescription(offer)
→ createAnswer()
→ setLocalDescription(answer)
→ answer 전송

handleAnswer()
→ setRemoteDescription(answer)

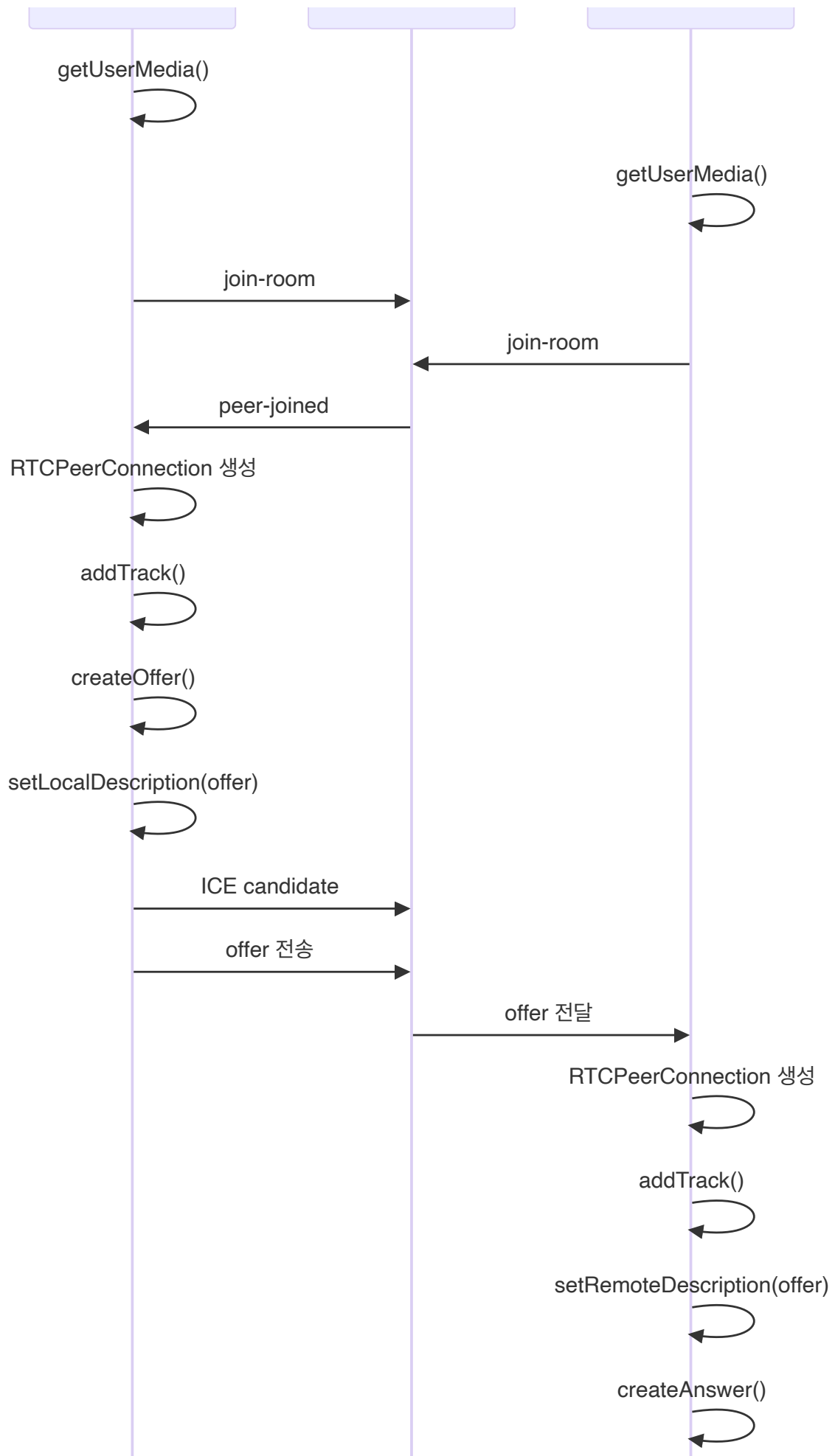
handleIceCandidate()
→ addIceCandidate()
```

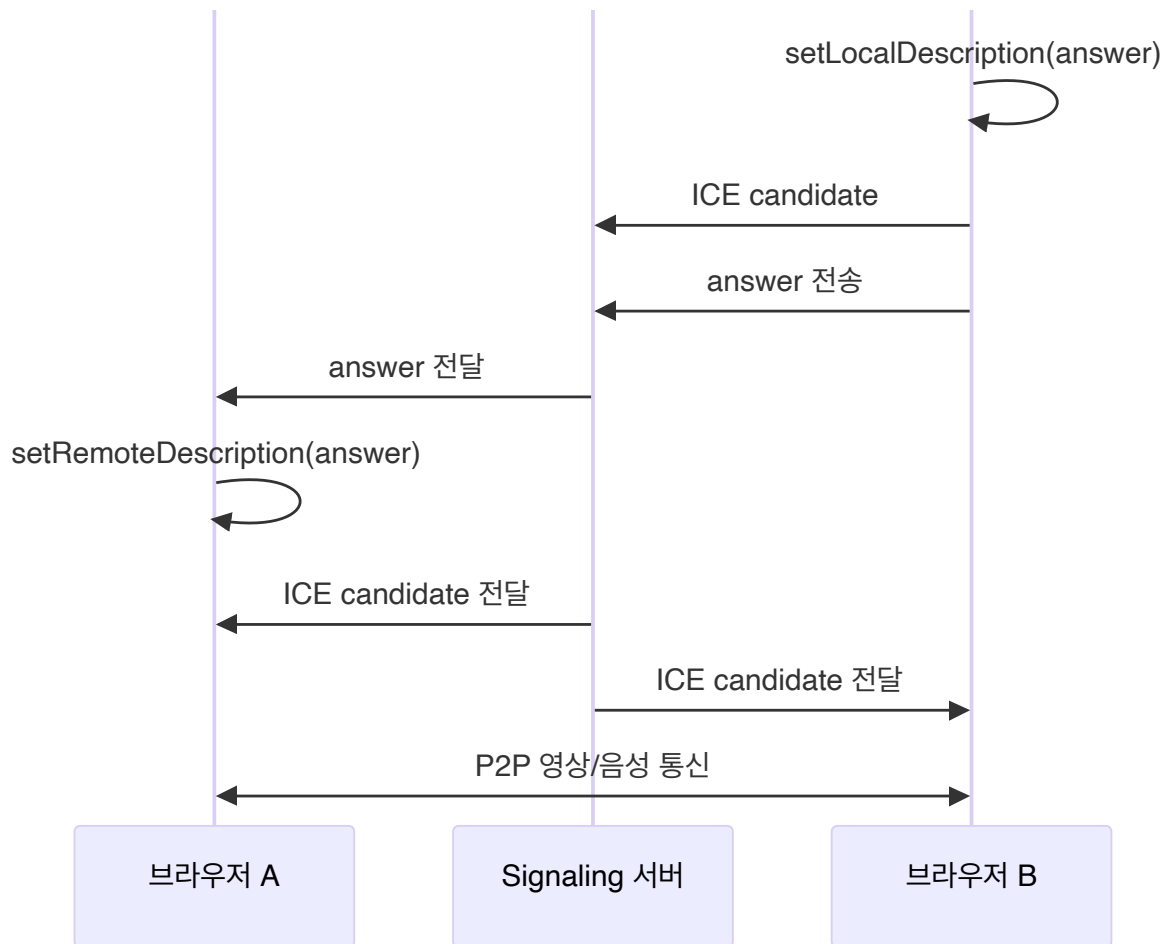
18. 전체 동작 시퀀스

브라우저 A

Signaling 서버

브라우저 B





19. 자주 헷갈리는 내용

Signaling 서버가 영상도 전달하나요?

아니요.

Signaling 서버는 연결 정보만 전달합니다.

```

offer
answer
ICE candidate
  
```

영상과 음성은 연결 성공 후 WebRTC PeerConnection을 통해 이동합니다.

STUN 서버가 미디어를 중계하나요?

아니요.

STUN 서버는 외부 주소를 알려주는 역할입니다.

미디어를 중계하지 않습니다.

TURN 서버는 언제 쓰나요?

브라우저끼리 직접 연결이 안 될 때 사용합니다.

TURN 서버는 실제 영상과 음성을 중계할 수 있습니다.

Offer와 Answer는 왜 필요한가요?

두 브라우저가 어떤 방식으로 통신할지 합의해야 하기 때문입니다.

코덱, 미디어 종류, 암호화 정보 등을 맞추는 과정입니다.

ICE Candidate는 왜 여러 개 생기나요?

브라우저가 가능한 여러 네트워크 경로를 찾기 때문입니다.

예:

로컬 네트워크 경로
STUN으로 얻은 외부 경로
TURN 서버 중계 경로

20. 발표용 핵심 문장

발표에서 그대로 사용할 수 있는 문장입니다.

WebRTC는 브라우저 간 실시간 영상, 음성, 데이터 통신을 가능하게 하는 기술입니다.

WebRTC는 연결 이후에는 P2P 방식으로 데이터를 주고받지만, 연결을 시작하기 위해서는 Signaling 서버가 필요합니다.

Signaling 서버는 영상과 음성을 직접 중계하지 않고, offer, answer, ICE candidate 같은 연결 정보를 전달합니다.

RTCPeerConnection은 WebRTC 연결을 관리하는 핵심 객체입니다.

Offer는 연결을 시작하는 쪽이 만드는 제안이고, Answer는 그 제안에 대한 응답입니다.

ICE Candidate는 두 브라우저가 실제로 연결 가능한 네트워크 경로 후보입니다.

STUN 서버는 외부에서 보이는 내 주소를 찾는 데 사용되고, TURN 서버는 직접 연결이 실패했을 때 미디어를 중계합니다.

21. 한 장 요약

WebRTC

→ 브라우저 간 실시간 통신 기술

P2P

→ 서버가 아니라 브라우저끼리 직접 통신

Signaling 서버

→ 연결 정보를 전달하는 서버

RTCPeerConnection

→ 실제 WebRTC 연결을 관리하는 객체

Offer

→ 연결 제안

Answer

→ 연결 응답

ICE Candidate

→ 연결 가능한 네트워크 경로 후보

STUN

→ 외부에서 보이는 내 IP/PORT 확인

TURN

→ 직접 연결 실패 시 미디어 중계